МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи

з дисципліни «Високорівневі мови програмування та фреймворки»

З лабораторної роботи No1

Виконали:

ст. гр. ПЗПІ-23-10

Шахмаєв Д.О.

Кисельов О.Д.

Прийняв:

асистент Кафедри ПІ

Саманцов О.О.

Харків 2026

Мета роботи

Ознайомитися з основами розробки вебдодатків на мові Python з використанням фреймворку Django. Вивчити архітектурний патерн MVC та особливості його реалізації в Django для розділення логіки даних, інтерфейсу та управління. Навчитися створювати проєкти та додатки, проектувати моделі даних за допомогою Django ORM, розробляти контролери (views) для обробки HTTP-запитів та створювати динамічні сторінки за допомогою системи шаблонів.

Хід роботи

1. Налаштування проєкту та створення структури

На першому етапі було розгорнуто робоче оточення для проєкту Django. Після перевірки наявності мови Python у системі, за допомогою пакетного менеджера pip було встановлено фреймворк. Ініціалізація проєкту була виконана командою django-admin startproject, що дозволило створити базову структуру директорій.

```
(.venv) PS C:\Users\vusen\PycharmProjects\DjangoProject> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
(.venv) PS C:\Users\vusen\PycharmProjects\DjangoProject> pip install Pillow
Collecting Pillow
  Downloading pillow-12.1.1-cp313-cp313-win_amd64.whl.metadata (9.0 kB)
Downloading pillow-12.1.1-cp313-cp313-win_amd64.whl (7.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 7.0/7.0 MB 14.0 MB/s eta 0:00:00
Installing collected packages: Pillow
Successfully installed Pillow-12.1.1
```

Рисунок 1 - Створення проекту

2. Створення суперкористувача (Admin User)

Для керування контентом через вбудований інтерфейс адміністратора було створено обліковий запис суперкористувача. За допомогою команди python manage.py createsuperuser було задано ім'я користувача, електронну адресу та пароль. Це забезпечило повний доступ до адмін-панелі Django, що дозволяє виконувати операції над моделями без написання додаткового коду на початковому етапі.

```
(.venv) PS C:\Users\vusen\PycharmProjects\DjangoProject> python manage.py createsuperuser
Username (leave blank to use 'vusen'):
Email address:
Password:
Password (again):
Error: Blank passwords aren't allowed.
Password:
Password (again):
Error: Your passwords didn't match.
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
```

Рисунок 2 - Створення суперкористувача

3. Робота з моделями даних (Models)

Для забезпечення збереження та обробки даних було створено відповідні моделі у файлі models.py. В Django модель представляє собою опис таблиці бази даних мовою Python. Основні аспекти реалізації: Кожна модель була описана як клас, успадкований від django.db.models.Model.Для зберігання інформації було обрано відповідні типи полів: CharField для текстових даних, IntegerField для оцінок та ImageField для завантаження фотографій. Для коректного відображення об'єктів в адмін-панелі було перевизначено метод __str__(), який повертає ПІБ та групу студента. Використання Django ORM дозволило уникнути ручного написання SQL-запитів, оскільки фреймворк автоматично відображає атрибути класу на колонки таблиці.

4. Налаштування та активація бази даних

Оскільки у файлі settings.py за замовчуванням визначено використання системи SQLite3, створення бази даних відбулося автоматично у вигляді

файлу db.sqlite3 у корені проєкту. Активація структури бази даних була виконана шляхом створення та застосування міграцій.

```
(.venv) PS C:\Users\vusen\PycharmProjects\DjangoProject> python manage.py makemigrations
Migrations for 'Lab1_vmptf':
  Lab1_vmptf\migrations\0001_initial.py
    + Create model Student
(.venv) PS C:\Users\vusen\PycharmProjects\DjangoProject> python manage.py migrate
Operations to perform:
  Apply all migrations: Lab1_vmptf, admin, auth, contenttypes, sessions
Running migrations:
  Applying Lab1_vmptf.0001_initial... OK
(.venv) PS C:\Users\vusen\PycharmProjects\DjangoProject> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
March 13, 2026 - 21:33:32
Django version 6.0.3, using settings 'DjangoProject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
```

Рисунок 3 - Налаштування та активація баз данних

5. Перевірка панелі адміністратора та наповнення бази даних

Після створення суперкористувача було проведено перевірку працездатності адміністративного інтерфейсу.

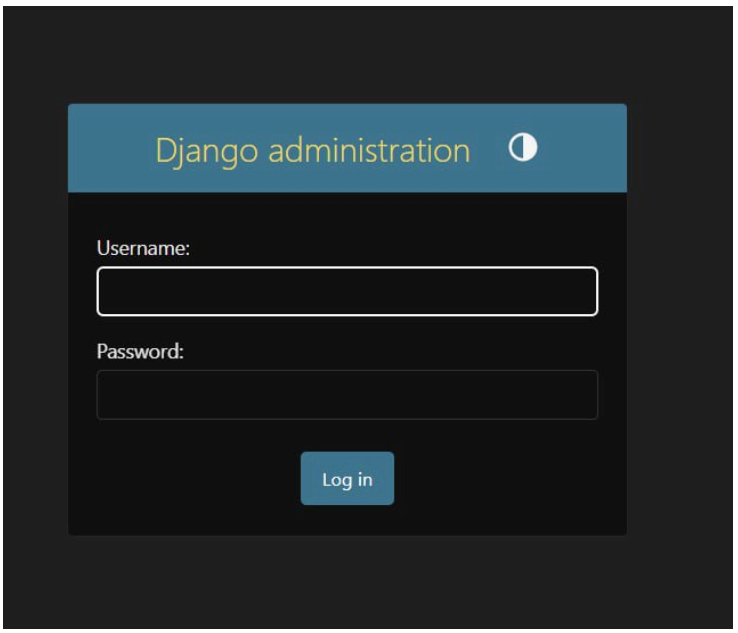


Рисунок 4 - Авторизація

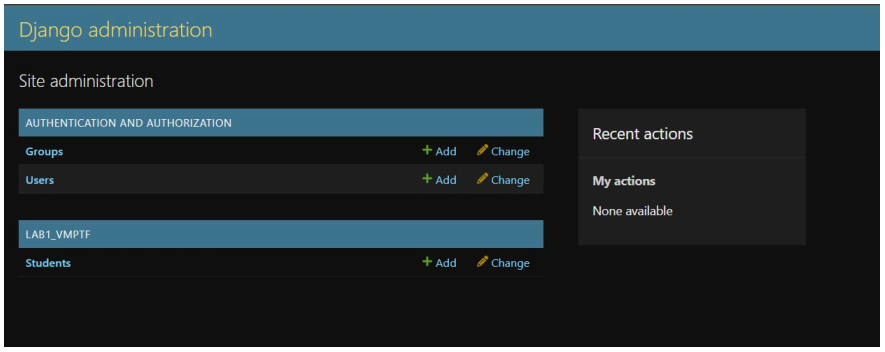


Рисунок 4 - Візуалізація адмін панелі

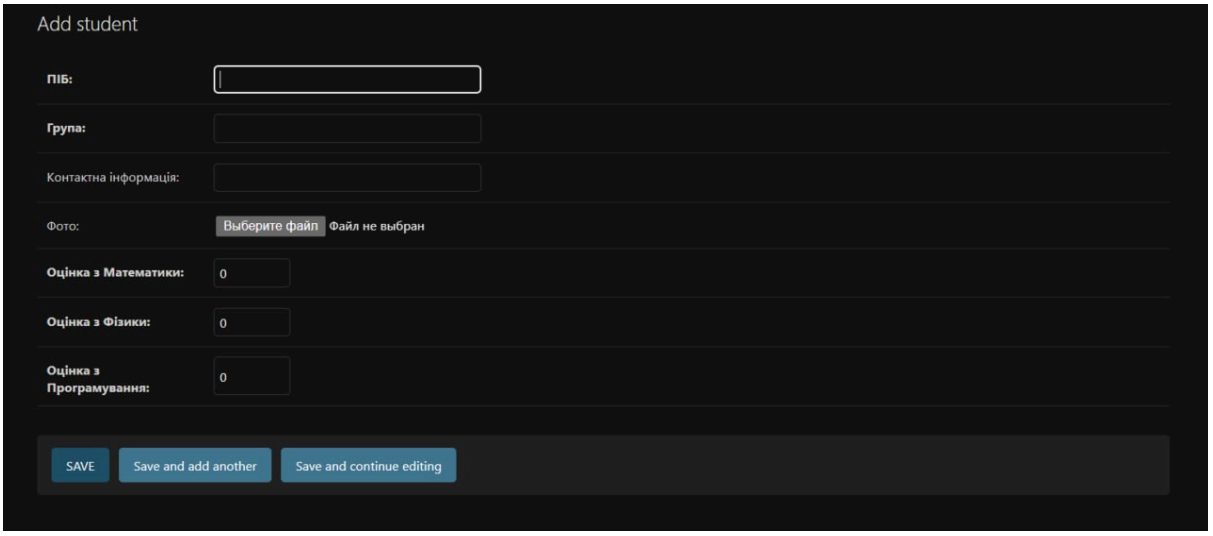


Рисунок 5 - Форма додавання студентів

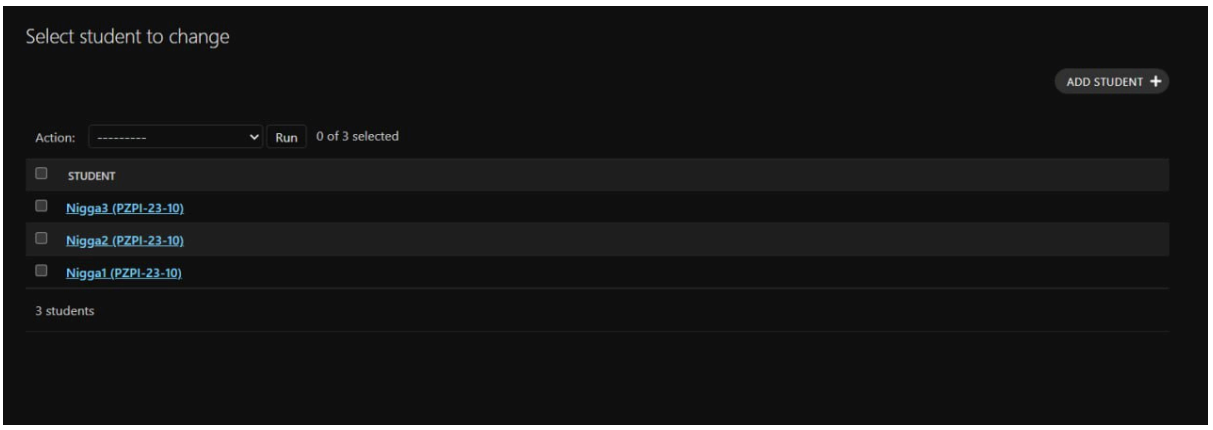


Рисунок 6 - Перегляд студентів

6. Розробка представлення

Для реалізації логіки передачі даних із бази на веб-сторінку у файлі views.py було створено функцію-контролер student_list. Дана функція отримує всі об'єкти моделі Student та передає їх у шаблон через словник контексту.

7. Налаштування маршрутизації

Для визначення адрес, за якими доступний список студентів, було налаштовано файл urls.py. Головну сторінку проєкту було пов'язано з функцією student_list. Також було додано налаштування для коректного відображення медіа-файлів (фотографій) під час розробки за умови активації режиму DEBUG.

8. Створення HTML-шаблону відображення

Для візуалізації даних було використано систему шаблонів Django. Шаблон дозволяє динамічно виводити інформацію за допомогою змінних та тегів логіки.

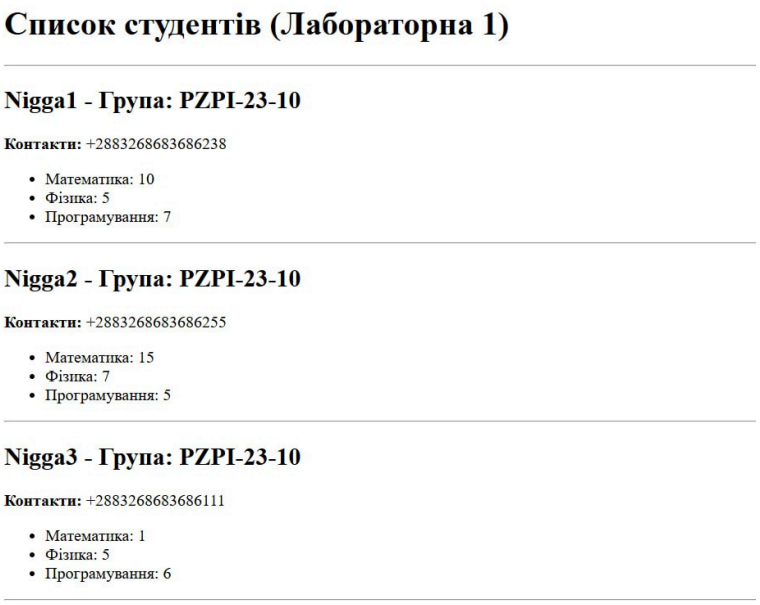


Рисунок 7 - Вивід студентів

9. Реалізація імпорту даних із CSV

Для автоматизації наповнення бази було впроваджено функціонал імпорту списку студентів із файлів формату .csv.

Розроблено функцію import_csv, яка зчитує файл, декодує дані та створює записи в базі даних за допомогою методу Student.objects.create.

Реалізовано обробку винятків для ігнорування некоректних рядків даних.

До шаблону було додано форму завантаження файлів із використанням атрибуту enctype="multipart/form-data" та захисного токена {% csrf_token %}.

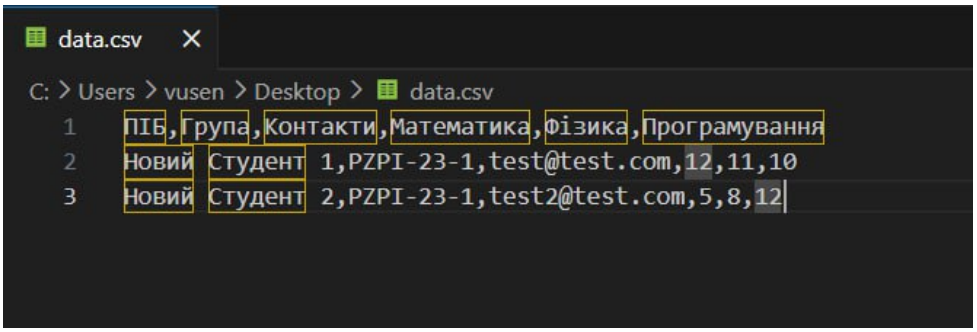


Рисунок 8 - csv файл з новими студентами

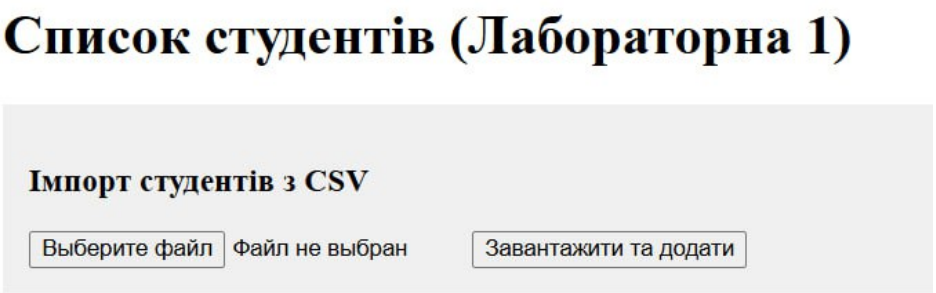


Рисунок 9 - Імпорт csv файлу

Новий Студент 1 - Група: PZPI-23-1

Контакти: test@test.com

- Математика: 12
- Фізика: 11
- Програмування: 10

Новий Студент 2 - Група: PZPI-23-1

Контакти: test2@test.com

- Математика: 5
- Фізика: 8
- Програмування: 12

Рисунок 10 - Відображення студентів

10. Створення методу для генерації звітів успішності (views.py)

Для виконання завдань високого рівня складності було реалізовано функціонал аналітики даних. У файлі views.py розроблено функцію student_report, яка виконує обробку статистичної інформації про успішність студентів.

Основні етапи обробки даних:

Розрахунок середнього бала: Для кожного об'єкта Student динамічно обчислюється середнє арифметичне значення оцінок з математики, фізики та програмування. Отриманий результат округлюється до двох знаків після коми.

Сортування: Отриманий список студентів автоматично сортується за допомогою методу sort у порядку спадання середнього бала (від найбільш успішних до найменш успішних).

Передача контексту: Результати обчислень передаються в окремий спеціалізований шаблон report.html.

11. Створення візуальної розмітки таблиці успішності

Для коректного та наочного відображення звіту було розроблено окремий HTML-шаблон.

Особливості реалізації інтерфейсу:

Стилізація (CSS): Додано вбудовані стилі для формування таблиці з чіткими межами, акцентованим заголовком (зелений колір) та чергуванням кольорів рядків для покращення читабельності даних.

Структура даних: Таблиця відображає ПІБ студента, його групу, бали з усіх предметів та підсумковий середній бал, виділений жирним шрифтом.

Навігація: Реалізовано посилання для швидкого повернення до загального списку студентів, що забезпечує зручність використання інтерфейсу.

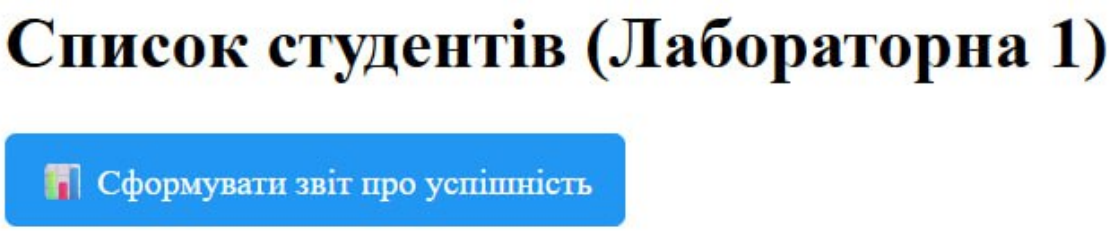
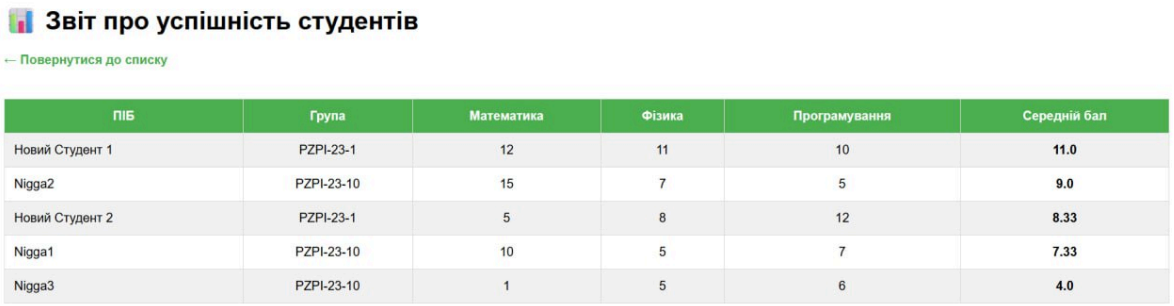


Рисунок 11 - Кнопка формування звіту успішності



ПІБ	Група	Математика	Фізика	Програмування	Середній бал
Новий Студент 1	PZPI-23-1	12	11	10	**11.0**
Nigga2	PZPI-23-10	15	7	5	**9.0**
Новий Студент 2	PZPI-23-1	5	8	12	**8.33**
Nigga1	PZPI-23-10	10	5	7	**7.33**
Nigga3	PZPI-23-10	1	5	6	**4.0**

Рисунок 12 - Формування звіту успішності

Висновки

Під час виконання лабораторної роботи було опановано основи розробки вебдодатків на мові Python з використанням фреймворку Django. У ході дослідження було детально вивчено архітектурний патерн MVC (Model-View-Controller) та його практичне застосування для розділення логіки даних, інтерфейсу та управління.

Контрольні запитання

1 Переваги Django: Висока швидкість розробки, принцип DRY ("не повторюйся"), вбудована адмін-панель, висока безпека "з коробки" та величезна кількість готових бібліотек.

2 Структура проєкту: Проєкт складається з головної папки конфігурації (налаштування, маршрути) та одного або кількох додатків (apps), кожен з яких відповідає за свою логіку (моделі, представлення, шаблони).

3 Основні компоненти:

- Моделі: Опис структури даних.
- URL-адреси: Маршрутизація запитів.
- Представлення (Views): Логіка обробки запитів та формування відповідей.
- Шаблони (Templates): Візуальне відображення даних користувачу.

4 Налаштування та запуск: Потрібно встановити Python та Django, створити проєкт через django-admin startproject, виконати міграції та запустити сервер командою python manage.py runserver.

5 Розгортання в продакшен: Використовується WSGI/ASGI сервер (наприклад, Gunicorn), веб-сервер (Nginx) для статики, налаштування DEBUG = False та захищена база даних (PostgreSQL).

6 Створення моделі: Створюється клас у models.py, що успадковується від models.Model, де описуються поля (CharField, IntegerField тощо).

7 Можливості ORM: Дозволяє працювати з БД за допомогою об'єктів Python замість написання SQL-запитів, автоматично створює таблиці та зв'язки.

8 Використання ORM: Через методи менеджера об'єктів, наприклад: Model.objects.all() для отримання всіх записів або Model.objects.create() для додавання нових.

9 URL для різних запитів: Вказуються у urls.py за допомогою path() або re_path(). Тип запиту зазвичай перевіряється всередині представлення через request.method.

10 Динамічні сторінки: Використовується мова шаблонів Django (DTL) з тегами {% %} для логіки та змінними {{ }} для виводу даних із контексту.

11 Клас Form: Дозволяє автоматично генерувати HTML-форми, проводити валідацію введених даних та перетворювати їх у типи Python.

12 Сторонні пакети: Встановлюються через pip та додаються до списку INSTALLED_APPS у файлі settings.py.

13 Middleware: Це "прошарок" між запитом та відповіддю, який дозволяє обробляти дані до того, як вони потраплять у представлення, або після того, як вийде відповідь (наприклад, для безпеки чи сесій).

14 Інструменти тестування: Вбудований фреймворк на базі unittest, клас TestCase та тестовий клієнт для імітації запитів.

15 Автентифікація та авторизація: Використовується вбудований додаток django.contrib.auth, який надає моделі користувачів, групи, права доступу та готові форми для входу.

16 Кешування: Django підтримує кешування на рівні окремих сторінок, частин шаблонів або всього сайту за допомогою Redis, Memcached або бази даних.

17 Система адміністрування: Налаштовується у файлі admin.py, де моделі реєструються через admin.site.register(ModelName).

18 Створення API: Найчастіше використовується сторонній пакет Django REST Framework (DRF), який додає серіалізатори та спеціальні представлення для JSON-відповідей.